



Fisher-Orthogonal Parameter Manifolds: Enforcing Hard Constraints in Hypernetwork-based Continual Learning



Jakub Michałowski

Department of Cognitive Science and Artificial Intelligence, School of Humanities and Digital Sciences

Introduction

Continual learning asks whether a neural network can learn a sequence of tasks without forgetting what it learned before. In practice, networks fail catastrophically: gradient descent on a new task overwrites weights that encoded prior knowledge, collapsing old-task accuracy within just a few update steps.

The field has produced three families of fixes — **regularization** (penalize changes to important weights), **replay** (store and re-train on old examples), and **architecture** methods (partition capacity per task) — but none fully solve the problem in a principled, parameter-efficient way. Most methods that project gradients do so in Euclidean space, which ignores the output-distribution geometry of the parameter space (e.g., OGD; Farajtabar et al., 2020), which fails to capture how a distribution-producing machine actually functions. By ignoring the sensitivity of the output to weight changes, these methods rely on an unreliable notion of importance. To fix this, I propose exploring second-derivative regularizers that respect the true geometry of the parameter space.

FOPNG: Fisher-Orthogonal Projected Natural Gradient Descent

Fisher Information and Natural Gradient

The **Fisher information matrix** $\mathcal{F}_\theta$ measures how sensitively the model's output distribution responds to each parameter:

$$\mathcal{F}_\theta = \mathbb{E}[\nabla_\theta \log p(y|x; \theta) \nabla_\theta \log p(y|x; \theta)^\top] \quad (1)$$

Standard gradient descent ignores this curvature. **Natural gradient descent** (Amari, 1998) corrects it by preconditioning with $\mathcal{F}_\theta^{-1}$, yielding updates that are equal-sized in distribution space and reparameterization-invariant:

$$\Delta\theta_{\text{nat}} = -\eta \mathcal{F}_\theta^{-1} \nabla_\theta L \quad (2)$$

*

References

- [1] Ishir Garg, Neel Kolhe, Andy Peng, and Rohan Gopalam. Fisher-orthogonal projected natural gradient descent for continual learning.
- [2] Marvin Minsky. Steps toward artificial intelligence. 49(1):8–30.
- [3] Johannes von Oswald, Christian Henning, Benjamin F. Grewe, and João Sacramento. Continual learning with hypernetworks.
- [4] Vinod Kumar Chauhan, Jiandong Zhou, Ping Lu, Soheila Molaei, and David A. Clifton. A brief review of hypernetworks in deep learning. 57(9):250.
- [5] Mehrdad Farajtabar, Navid Azizan, Alex Mott, and Ang Li. Orthogonal gradient descent for continual learning.
- [6] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. 114(13):3521–3526.
- [7] Neil De La Fuente, Maria Pilligua, Daniel Vidal, Albin Soutiff, Cecilia Curreli, Daniel Cremers, and Andrey Barsky. Prototype augmented hypernetworks for continual learning.
- [8] Hamed Hemati, Vincenzo Lomonaco, Davide Bacciu, and Damian Borth. Partial hypernetworks for continual learning.
- [9] Benjamin Ehret, Christian Henning, Maria R. Cervera, Alexander Meulemans, Johannes von Oswald, and Benjamin F. Grewe. Continual learning in recurrent neural networks.
- [10] Stephen Grossberg. How does a brain build a cognitive code? 87(1):1–51. Place: US Publisher: American Psychological Association.
- [11] SueYeon Chung, Daniel D. Lee, and Haim Sompolinsky. Classification and geometry of general perceptual manifolds. 8(3):031003.
- [12] Alex Krizhevsky. Learning multiple layers of features from tiny images.
- [13] David Ha, Andrew Dai, and Quoc V. Le. HyperNetworks.
- [14] Ari S. Benjamin, David Rolnick, and Konrad Kording. Measuring and regularizing networks in function space.

Orthogonal Projection

After each task k , the gradient g_k is stored in a memory matrix $G = [g_1 \mid \dots \mid g_{t-1}]$. New updates are projected onto the **Fisher-orthogonal complement** of G — directions that cannot alter prior task outputs:

$$\Delta\theta = -\eta (I - G(G^\top \mathcal{F}_\theta G)^{-1} G^\top \mathcal{F}_\theta) \mathcal{F}_\theta^{-1} \nabla_\theta L_t \quad (3)$$

The term $G(G^\top \mathcal{F}_\theta G)^{-1} G^\top \mathcal{F}_\theta$ is a projection that reads off and removes any component of the new gradient that would disturb old tasks. What remains is a geometrically hard-constrained update.

Combining those two ideas gives us **FOPNG** (Garg et al., 2026, preprint), which I adapt to the hypernetwork setting.

Hypernetworks

A **hypernetwork** $h(\cdot; \phi)$ generates the weights of a target network rather than making predictions itself. Each task t receives a learned embedding z_t , producing task-specific weights on the fly:

$$\theta_t = h(z_t; \phi) \quad (4)$$

The shared parameters ϕ do not grow with the number of tasks, and each task gets a fully specialized network by varying z_t . This approach was first applied to continual learning by von Oswald et al. (2020). Applying FOPNG to ϕ means the Fisher matrix and gradient memory are maintained only for this compact set of parameters, not the larger target network.

For very large target networks, **chunking** (Ha et al., 2017) keeps ϕ tractable: the hypernetwork generates target weights in small blocks, each conditioned on z_t and a chunk index, rather than all at once.

Baselines

SGD and **Adam** are naive baselines with no forgetting protection, establishing the lower bound.

EWC (Elastic Weight Consolidation; Kirkpatrick et al., 2017) penalizes changes to weights deemed important for prior tasks via the diagonal Fisher, but operates in Euclidean space.

* All results are preliminary and subject to change.

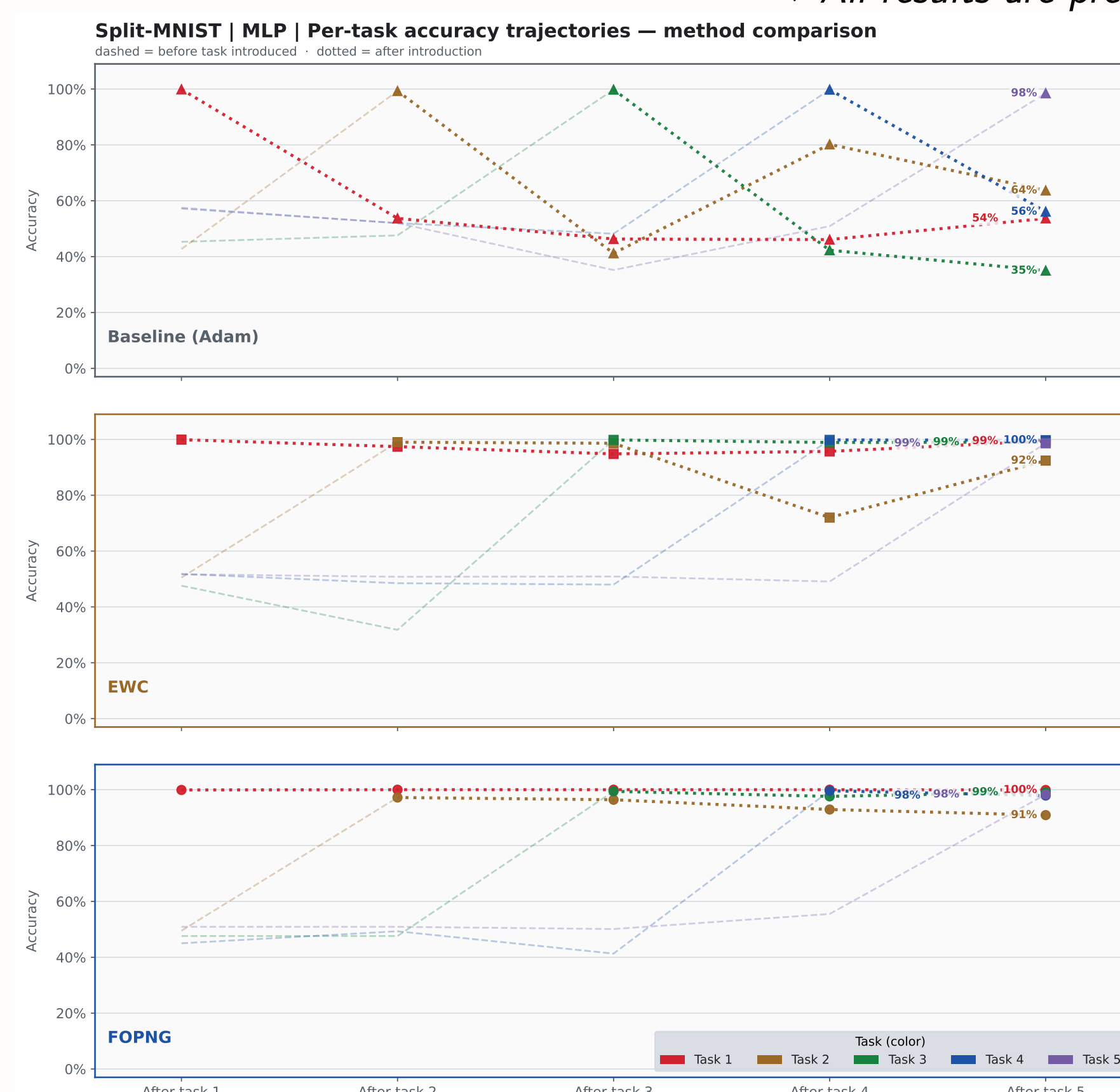


Figure 1: First preliminary results for Split-MNIST. Standard MLP performance comparison. Notice the severe performance degradation in Baseline (Adam).

OGD (Orthogonal Gradient Descent; Farajtabar et al., 2020) projects gradients onto the Euclidean orthogonal complement of prior gradients — the closest Euclidean analogue to FOPNG.

FNG (Fisher Natural Gradient) applies natural gradient updates without orthogonal projection, isolating the contribution of Fisher preconditioning alone.

Benchmarks

Split-MNIST splits MNIST into 5 binary tasks (e.g. 0 vs 1, then 2 vs 3). The standard entry-level benchmark for sequential learning.

Permuted-MNIST applies a different random pixel permutation per task, all sharing the same label space. Tests long-horizon retention over 10+ tasks.

Split-CIFAR-10 and **Split-CIFAR-100** apply the split protocol to natural images: 5 tasks over 10 classes, and 20 tasks over 100 classes respectively. Higher visual complexity and more tasks make these substantially harder.

Evaluation Metrics

I will report **average accuracy** across all tasks after training, **backward transfer** (how much new training degrades old tasks), **forward transfer** (whether prior tasks help new ones), and **per-task accuracy trajectories** to visualize forgetting dynamics over time. I will also introduce many tangential metrics for deeper analysis.

Summary and Conclusions

This project combines FOPNG with Hypernetworks to enforce geometrically hard constraints on continual learning updates. By operating in the Fisher-Riemannian manifold rather than Euclidean space, the method leverages second-derivative information to preserve task-specific knowledge without storing raw data or expanding model capacity. Preliminary results on Split-MNIST are promising. Full evaluation across Permuted-MNIST and Split-CIFAR benchmarks is ongoing.

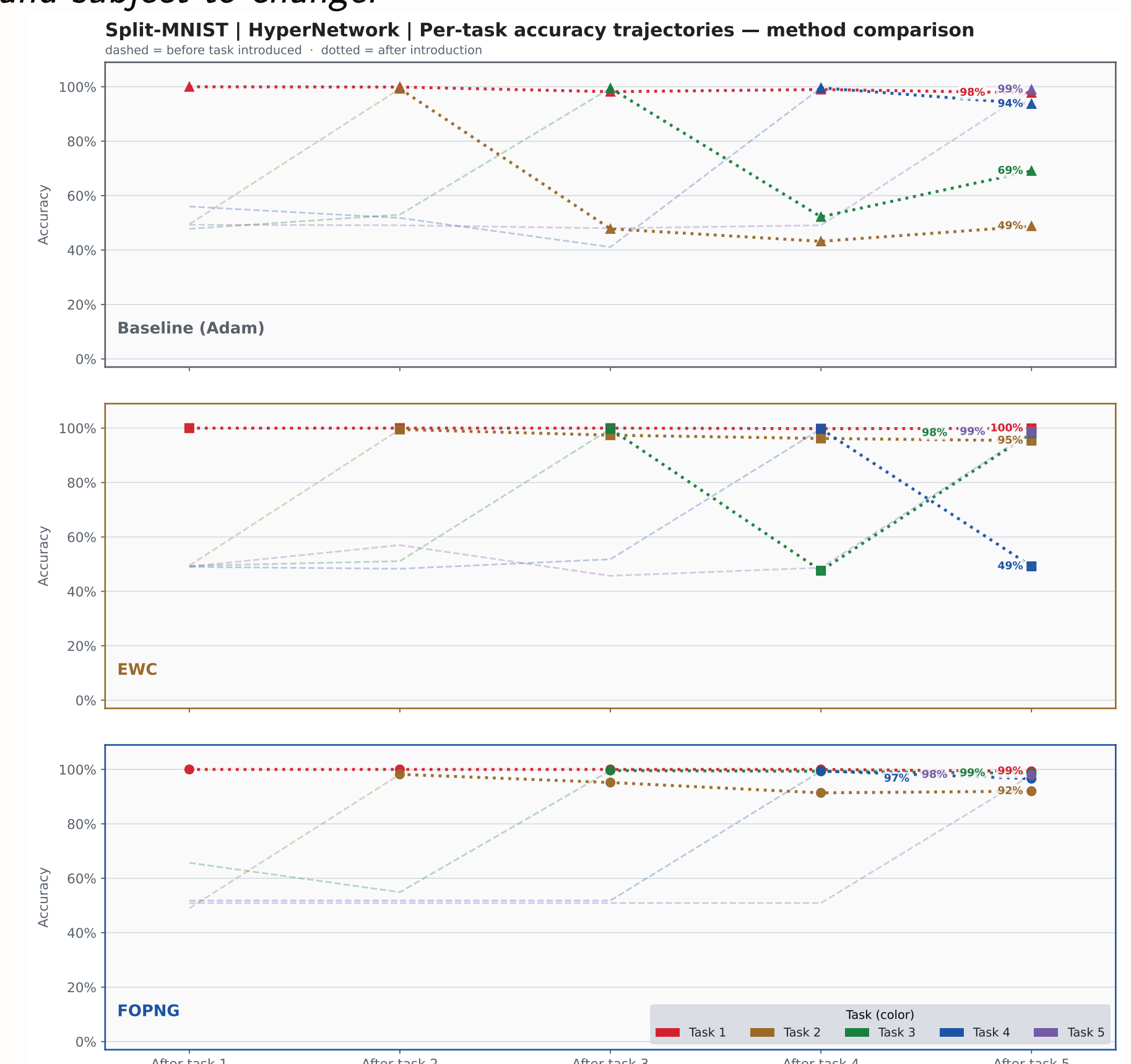


Figure 2: Full (not chunked) Hypernetwork performance on Split-MNIST. FOPNG successfully maintains high accuracy across all sequential tasks without catastrophic forgetting.